

FULL STACK SHOPEZ E-COMMERCE WEB APPLICATION (MERN)

1. Introduction

- **Project Title:** ShopEZ – MERN Stack E-Commerce Web Application
- **Team Members:**

SYED SHABEENA [Frontend Development, Backend Development, Database Design, Documentation]

SIRITHOTI SHULAMYTHI [Frontend Development, Testing]

2. Project Overview

Purpose:

ShopEZ is a full-stack e-commerce web application developed using the MERN stack (**MongoDB, Express.js, React.js, and Node.js**). The project provides customers with a simple and user-friendly platform to browse products, view product details, add products to the shopping cart, place orders, and manage their accounts. Administrators can manage products, banners, and customer orders through a secure admin dashboard.

Features:

Feature	Functionality
User Registration	Allows new users to create an account using their username, email, and password.
User Login	Authenticates registered users and provides secure access to the application.
Product Catalog	Displays products organized into different categories with attractive product cards.
Product Search	Enables users to search for products using the search bar.
Product Details	Displays detailed information about a selected product, including images, variants, price, and description.
Shopping Cart	Allows users to add products to the cart and manage selected items.
Order Placement	Enables users to place orders by providing delivery details.
Admin Login	Allows administrators to securely access the admin dashboard.
Product Management	Enables administrators to add, update, and delete products.

Banner Management	Allows administrators to update the homepage banner displayed to users.
Responsive User Interface	Provides a user-friendly interface that works across different screen sizes.

3. Architecture

The **ShopEZ E-Commerce Application** follows a **three-tier client-server architecture** developed using the **MERN Stack (MongoDB, Express.js, React.js, and Node.js)**. The architecture is divided into three layers: **Presentation Layer (Frontend)**, **Business Logic Layer (Backend)**, and **Data Layer (Database)**. This modular architecture improves scalability, maintainability, security, and performance. The React.js frontend communicates with the Express.js backend through RESTful APIs, while MongoDB stores all application data.

Frontend:

The frontend of **ShopEZ** is developed using **React.js** with **Vite**, providing a fast, responsive, and interactive user interface. The application follows a **component-based architecture**, where reusable components improve code reusability and simplify maintenance. Navigation between pages is managed using **React Router DOM**, while **Axios** is used to communicate with the backend REST APIs. CSS is used to create an attractive and responsive user interface across different devices.

The frontend provides dedicated interfaces for different users:

- User Interface
- Admin Dashboard

Frontend Responsibilities:

- User Registration and Login
- Product Browsing
- Product Search and Filtering
- Product Details View
- Shopping Cart Management
- Order Placement
- Order History
- User Profile Management
- Admin Product Management
- Responsive User Interface
- **Technologies Used**
- React.js
- Vite
- HTML5
- CSS3
- JavaScript (ES6)
- Axios
- React Router DOM

Backend:

The backend is developed using **Node.js** and **Express.js**. It follows the **MVC (Model–View–Controller)** architecture, which separates routing, business logic, and database operations for better maintainability and scalability.

The backend exposes **RESTful APIs** to process client requests, authenticate users, manage products, shopping carts, orders, banners, and administrator operations. Secure authentication is implemented using **JWT (JSON Web Token)**, while **bcryptjs** is used to encrypt user passwords before storing them in the database.

Backend Responsibilities

- User Authentication
- Admin Authentication
- JWT Token Generation and Verification
- Product Management
- Shopping Cart Management
- Order Management
- Banner Management
- User Management
- Product Search and Filtering
- Database Operations
- REST API Development

Technologies Used

- Node.js
- Express.js
- JWT (JSON Web Token)
- bcryptjs
- CORS
- dotenv
- Mongoose
-

Database:

The application uses MongoDB as the NoSQL database to store all application data. Database operations are performed using Mongoose ODM, which provides schema-based modeling, validation, and efficient interaction with MongoDB collections.

The primary collections used in the system are:

- Users
- Products
- Cart
- Orders
- Admin

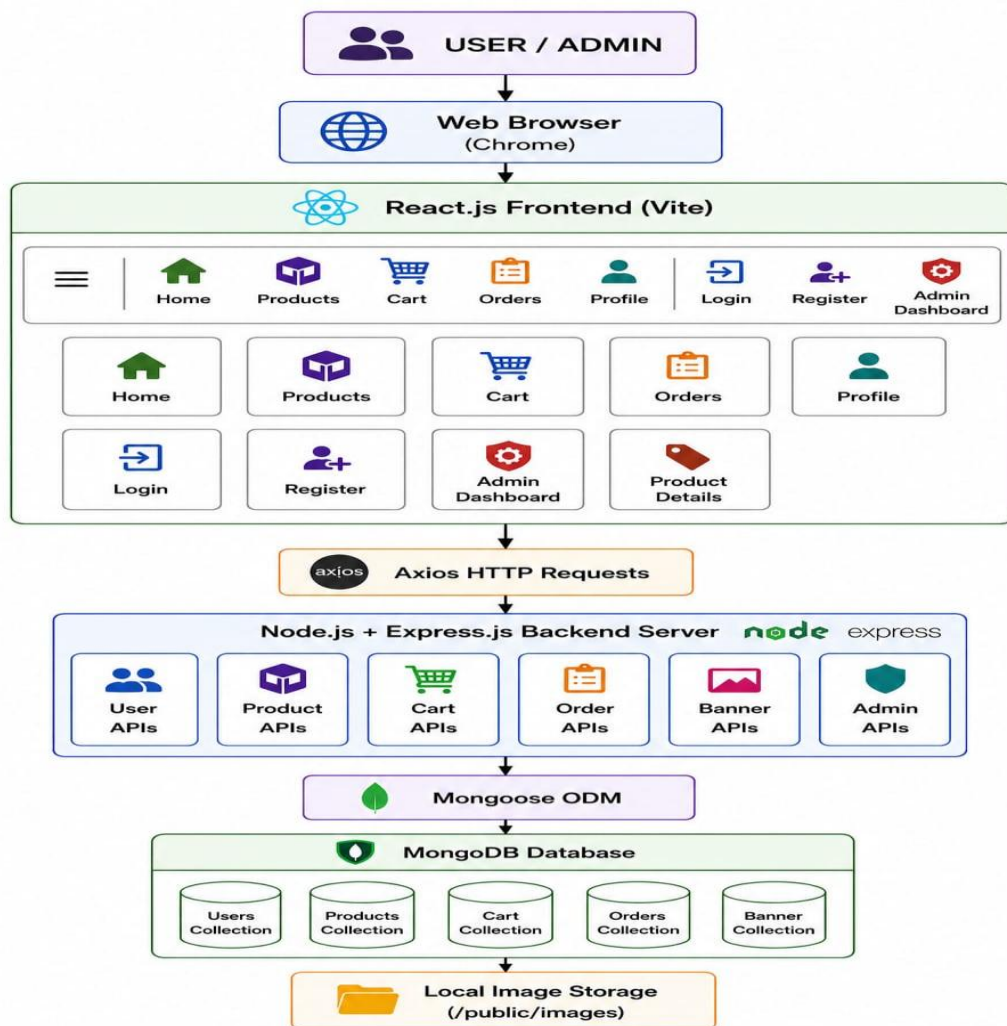
- Banners

Database Responsibilities:

- Store User Information
- Store Product Details
- Store Shopping Cart Items
- Store Customer Orders
- Store Administrator Information
- Store Homepage Banner Details
- Maintain Relationships Between Users, Products, Cart, and Orders

Technologies Used

- MongoDB
- MongoDB Compass
- Mongoose ODM



ShopEZ Solution Architecture

4. Setup Instructions

Prerequisites:

- Visual Studio Code
- Node.js (v18 or above)
- npm (Node Package Manager)
- MongoDB Compass (with Local MongoDB Server)
- Google Chrome
- Postman

Installation:

Step 1: Clone the Project

`git clone <repository-link>`

Step 2: Open the Project

Open the project folder using **Visual Studio Code**.

Step 3: Install Frontend Dependencies

Open Terminal:

`cd client`

`npm install`

This command installs all required frontend packages such as:

- React
- React Router DOM
- Axios
- Vite
- Other dependencies listed in package.json

Step 4: Install Backend Dependencies

Open another terminal: (split terminal)

`cd server`

`npm install`

This command installs all required backend packages such as:

- Express.js
- CORS
- dotenv
- bcryptjs
- jsonwebtoken
- MongoDB Driver (or Mongoose, if used)
- multer (if used)
- Other dependencies listed in package.json

Step 5: Start MongoDB

Open **MongoDB Compass**.

Start the **Local MongoDB Server** and ensure the database connection is active

before running the backend server.

Step 6: Configure Environment Variables

Create a .env file inside the **server** folder and configure the required environment variables.

Like:

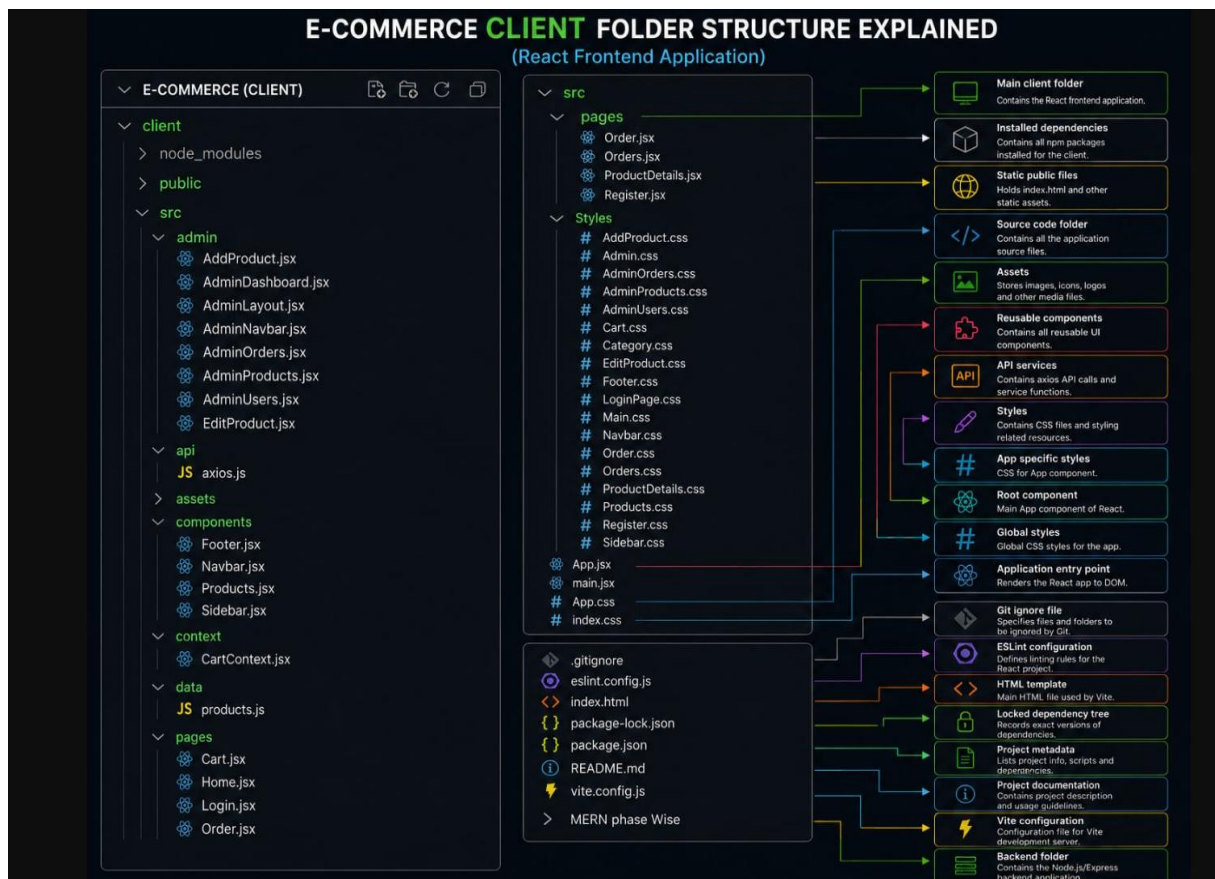
PORT=5000

MONGO_URI=your_mongodb_connection_string

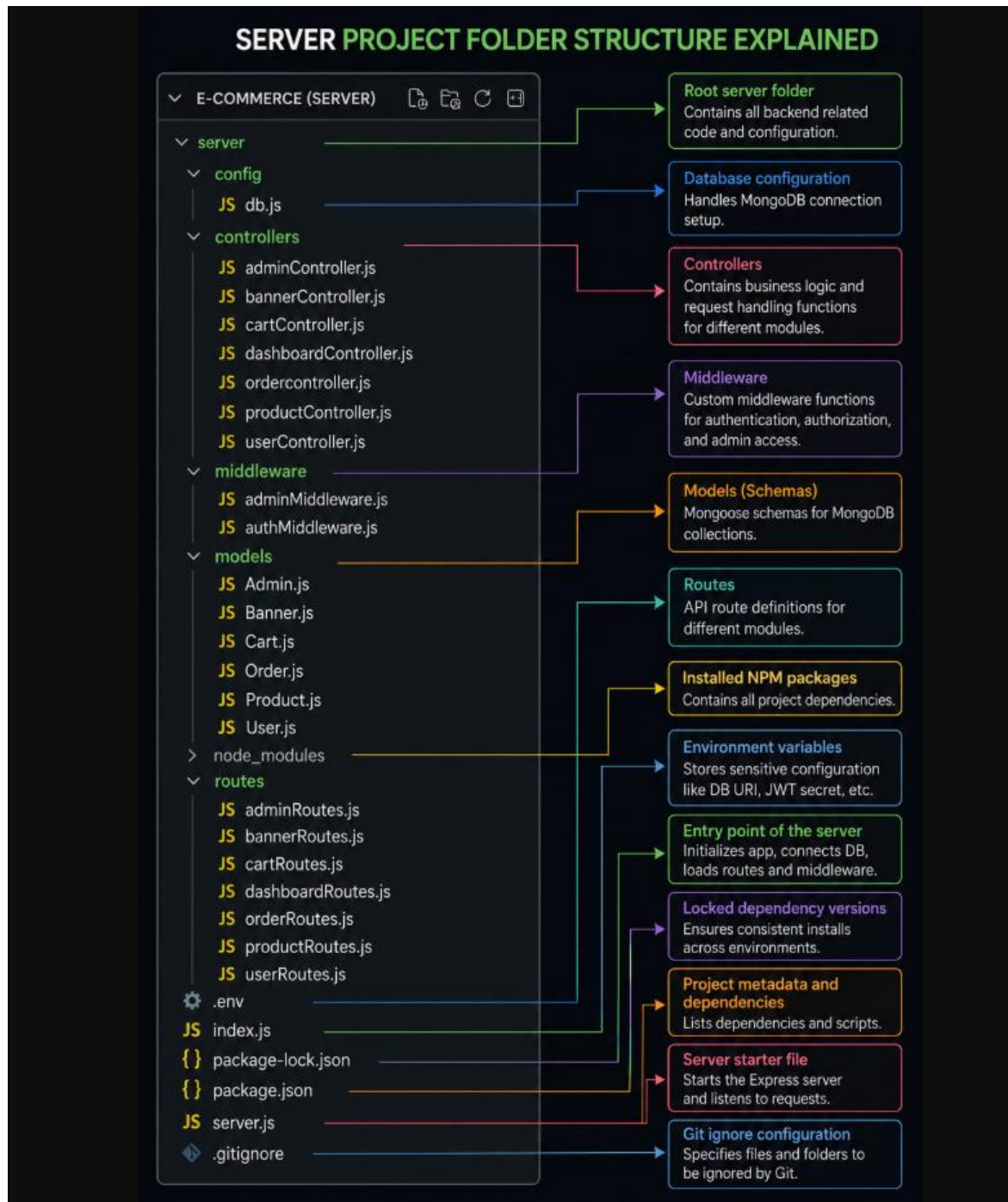
JWT_SECRET=your_jwt_secret key

5. Folder Structure

Client:



Server:



6. Running the Application

6.1 Start the Backend Server

Open a terminal and navigate to the server directory.

```
cd server
```

Start the backend server using the following command:

```
npm start
```

If the server starts successfully, the terminal displays:

MongoDB Connected Successfully

Server running on port 5000

The backend server will be available at:

http://localhost:5000

6.2 Start the Frontend Application

Open another terminal and navigate to the client directory.

cd client

Start the React application using Vite:

npm run dev

After successful execution, Vite displays:

VITE vX.X.X ready in XXX ms

→ Local: http://localhost:5173/

Open the following URL in your browser:

http://localhost:5173

7. API Documentation

User APIs:

Method	Endpoint	Parameters / Request Body	Description	Example Response
POST	/api/users/register	username, email, password	Registers a new user	{"message": "User registered successfully"}
POST	/api/users/login	email, password	Authenticates a user	{"token": "...", "user": {...}}

Admin APIs:

Method	Endpoint	Parameters / Request Body	Description	Example Response
POST	/api/admin/login	email, password	Authenticates a administrator	{"token": "...", "user": {...}}

Product APIs:

Method	Endpoint	Parameters / Request Body	Description	Example Response
GET	/api/products	None	Retrieves all products	[{ ... }, { ... }]
GET	/api/products/:id	Product ID	Retrieves a specific product	{ "_id": "...", "title": "Shirt", "price": 999 }
POST	/api/products	Product Details	Adds a new product	{ "message": "Product added successfully" }
PUT	/api/products/:id	Product ID + Updated Details	Updates an existing product	{ "message": "Product updated successfully" }
DELETE	/api/products/:id	Product ID	Deletes a product	{ "message": "Product updated successfully" }

Cart APIs:

Method	Endpoint	Parameters / Request Body	Description	Example Response
GET	/api/cart	None	Retrieves cart items	[{ ... }]
POST	/api/cart	user, product, selectedVariant, quantity	Adds a product to the cart	{ "message": "Product added to cart" }
DELETE	/api/cart/:id	Cart Item ID	Removes an item from the cart	{ "message": "Item removed successfully" }

Order APIs:

Method	Endpoint	Parameters / Request Body	Description	Example Response
POST	/api/orders	Customer & Order Details	Places a new order	{ "message": "Order" }

				placed successfully" }
GET	/api/orders	None	Retrieves all orders	[{...}, {...}]

Banner APIs:

Method	Endpoint	Parameters / Request Body	Description	Example Response
GET	/api/banner	None	Retrieves homepage banner	{ "image":"banner.jpg" }
POST	/api/banner	Banner Image	Updates homepage banner	{ "message":"Banner updated successfully" }

8. Authentication

Authentication and authorization in the ShopEZ application are implemented using **JSON Web Tokens (JWT)**. The application provides separate login mechanisms for users and administrators to ensure secure access to the system.

During user registration, passwords are securely encrypted using the **bcryptjs** library before being stored in the MongoDB database. During login, the entered password is verified against the encrypted password stored in the database. If the credentials are valid, the backend generates a **JWT token** using the **jsonwebtoken** library with a validity period of **7 days**.

A separate administrator login endpoint is also provided. After successful verification of the administrator's credentials, a JWT token is generated and returned along with the administrator's information.

The generated JWT token is stored in the browser's **Local Storage** and is included in the **Authorization** header as a **Bearer Token** when accessing protected API endpoints.

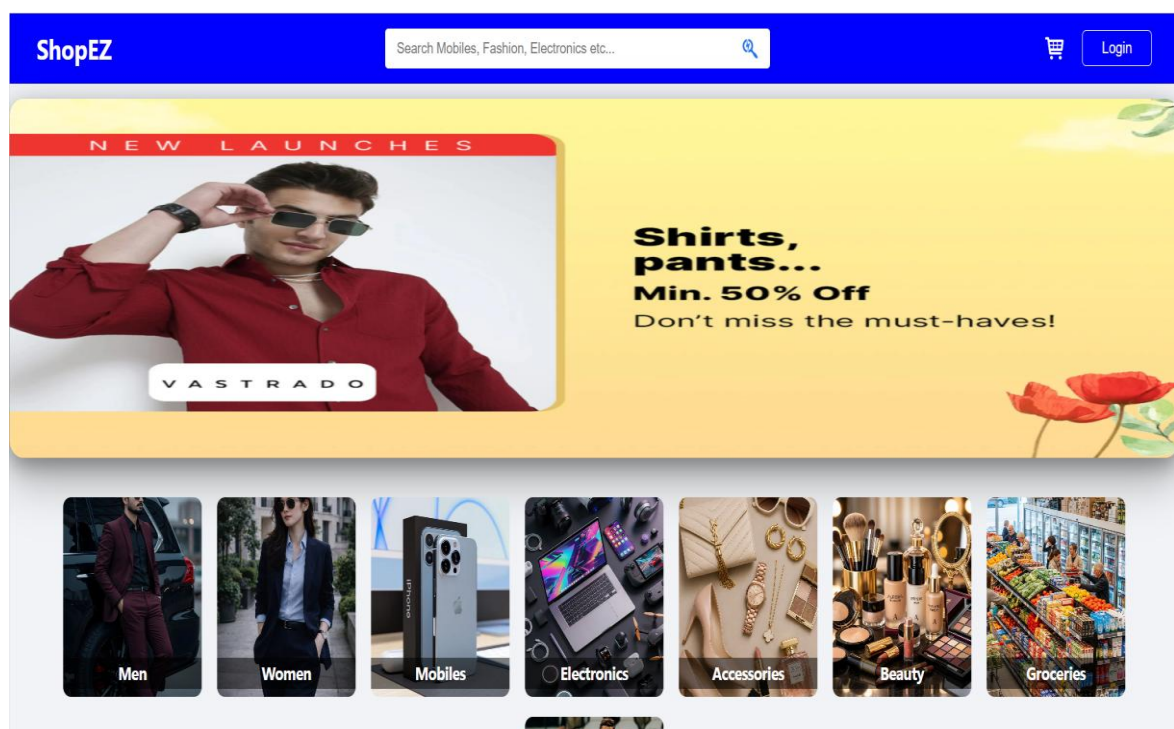
The application uses a custom **authentication middleware** to verify the JWT token before allowing access to protected routes such as cart management, order management, and product management. If the token is missing or invalid, the server returns an **HTTP 401 (Unauthorized)** response, preventing unauthorized access.

The ShopEZ application does not use server-side sessions. Instead, it relies on **JWT-based authentication and authorization**, providing a secure and scalable mechanism for protecting user and administrator functionalities.

9. User Interface:

The **ShopEZ E-Commerce Application** provides a modern, responsive, and user-friendly interface developed using **React.js** and **CSS3**. The application is designed to offer a smooth shopping experience for customers and an easy-to-use management interface for administrators. Navigation is simple and intuitive, allowing users to browse products, manage their shopping cart, and place orders efficiently.

Landing (Home) Page:



User Registration Page:

ShopEZ

Search Mobiles, Fashion, Electronics etc...

Login

Create Account

Full Name

Email

Password

Confirm Password

Register

Already have an account? [Login](#)

ShopEZ

Quick Links

Account

Categories

Support

User Login Page:

ShopEZ

Search Mobiles, Fashion, Electronics etc...

Login

Login

Email

Password

Sign In

Not registered? [Create an account](#)

ShopEZ

One Destination for all your needs.

Quick Links

Home

Categories

My Profile

Account

Cart

Profile

Order History

Categories

Electronics

Fashion

Beauty

Home & Kitchen

Support

Contact Us

Privacy Policy

Terms & Conditions

Products Page:

Filters

Sort By

☐ Popular
 ☐ Price (Low to High)
 ☐ Price (High to Low)
 ☐ Discount

Categories

☐ Men
 ☐ Women
 ☐ Mobiles
 ☐ Electronics
 ☐ Accessories
 ☐ Beauty
 ☐ Groceries
 ☐ Sports

Gender

All Products

VebNoR

Men Solid Brown Polo T-...

₹254 ₹999 **74% off**

Variant : M, L, XL

Only few left!

R R STUDIO

Women Fit and Flare Bro...

₹512 ₹1,799 **71% off**

Variant : S, M, L, XL, XXL, 3XL

FUSION FLY

3in1 Foldable Makeup Ey...

₹562 ₹1,499 **62% off**

Variant : Standard Pack

Only few left!

Apple

Apple iPhone 17 Pro (Cos...

₹144,900 ₹154,900 **6% off**

Variant : 128 GB, 256 GB, 512 GB

Samsung

Samsung Galaxy S23 Ultr...

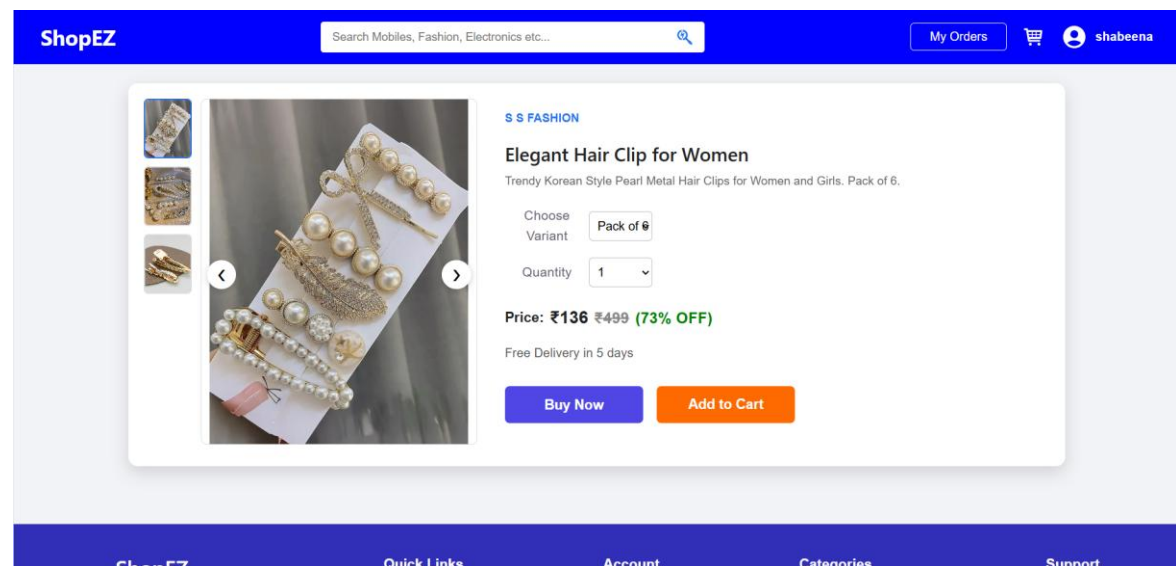
₹164,999 ₹174,999 **2% off**

Variant : 512 GB

ASUS

Vivobook S

Products Details Page:



9. Testing

The ShopEZ application was tested throughout the development process to ensure that all features function correctly and provide a smooth user experience. Both frontend and backend components were tested individually and together to verify the complete workflow of the application.

The testing strategy included **functional testing**, **integration testing**, and **manual user interface testing**. Functional testing was performed to verify user registration, login, product management, shopping cart operations, order placement, banner management, and administrator functionalities. Integration testing ensured proper communication between the React frontend, Express.js backend, and MongoDB database. Manual UI testing was conducted to verify navigation, responsiveness, form validations, and overall user experience.

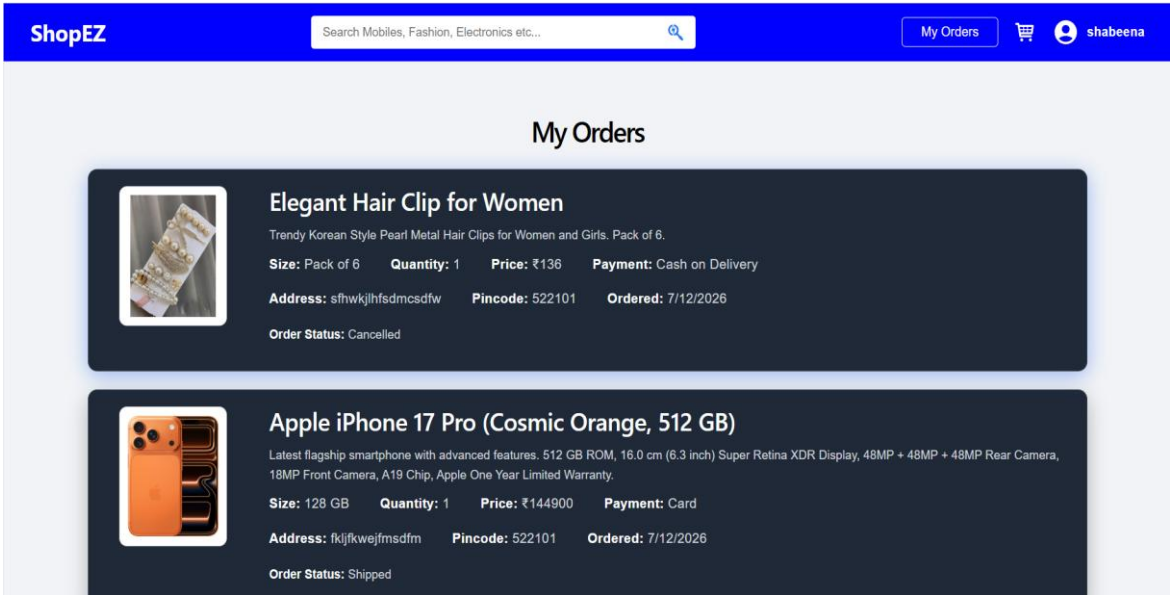
The following tools were used during testing:

- **Postman** – Used to test REST API endpoints such as user registration, login, product management, cart operations, order management, and banner management.
- **Google Chrome** – Used to test the frontend user interface, responsiveness, navigation, and overall application behavior.
- **MongoDB Compass** – Used to verify data storage, retrieval, and updates in the MongoDB database.
- **Node.js and Express.js Console Logs** – Used to monitor backend execution, API requests, and error messages during development.

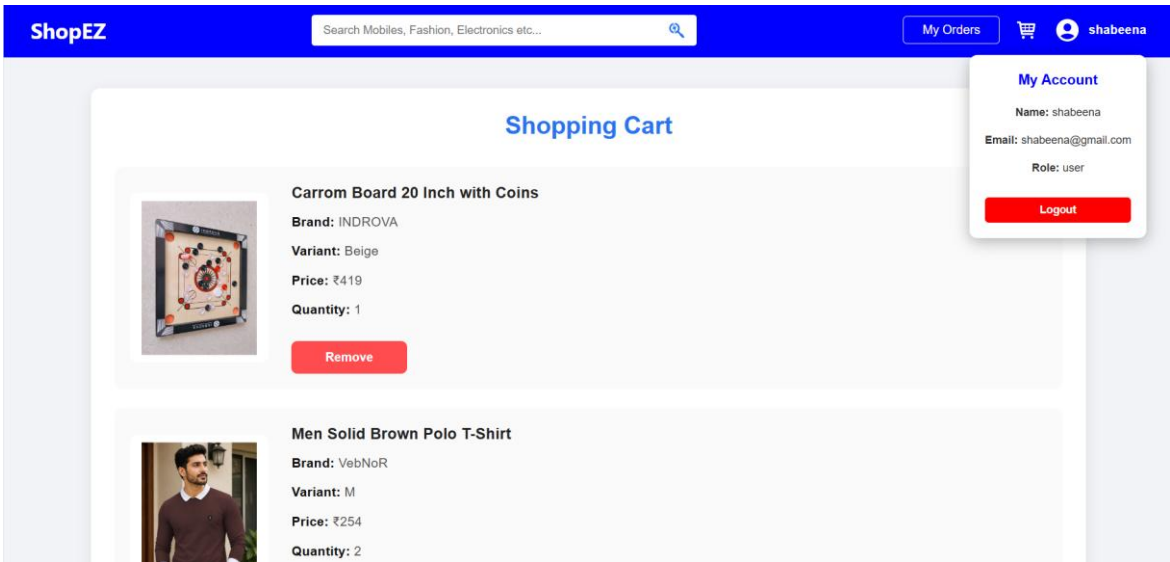
Testing confirmed that the core functionalities of the ShopEZ application operate correctly, including user authentication, product browsing, shopping cart management, order placement, and administrator operations.

10. Screenshots or Demo

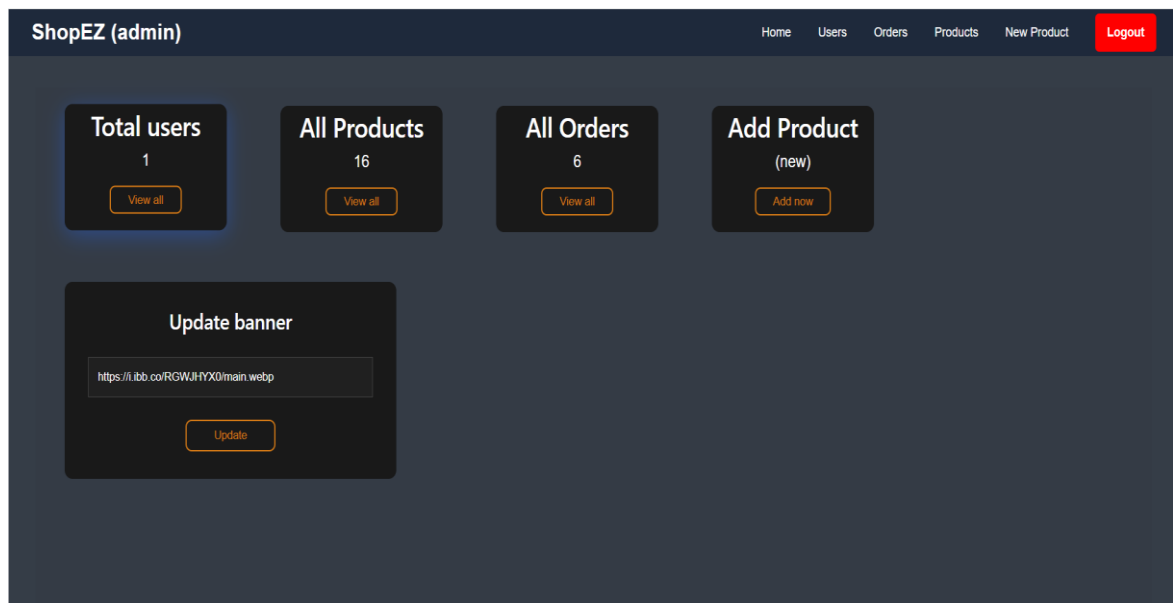
My Orders Page:



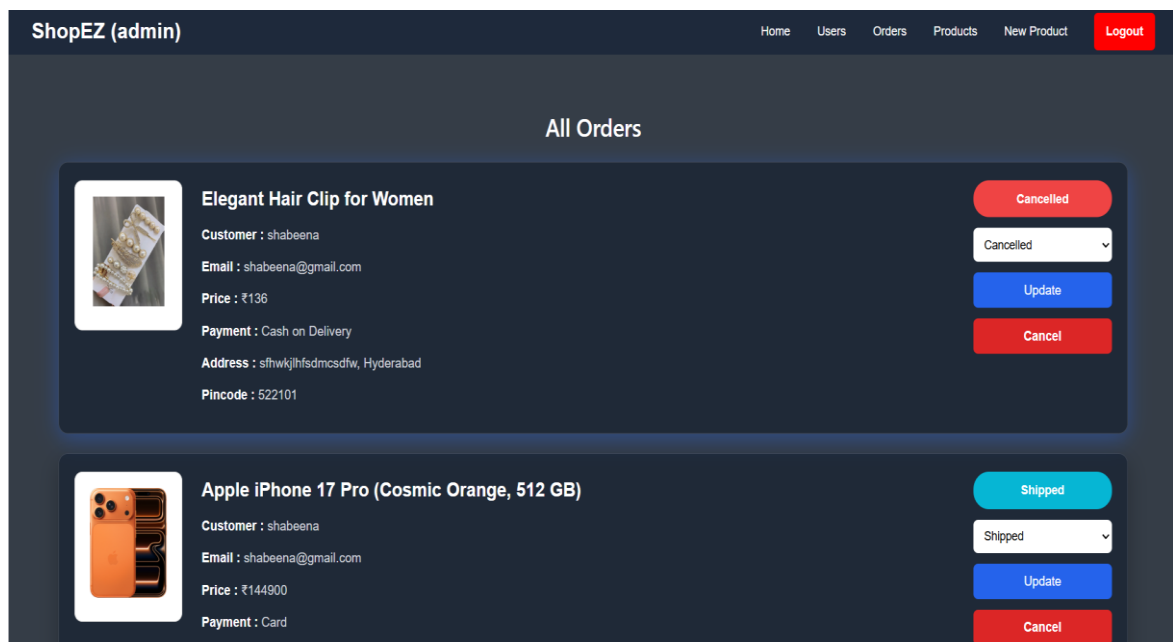
Shopping Cart Page:



Admin Page:



All Orders Page(Admin):



All Products Page:

ShopEZ (admin)

HomeUsersOrdersProductsNew ProductLogout

All Products

Image	Name	Brand	Price	Action
	Men Solid Brown Polo T-Shirt	VebNoR	₹254	Edit Delete
	Women Fit and Flare Brown Maxi/Full Length Dress	R R STUDIO	₹512	Edit Delete
	3in1 Foldable Makeup Eyeshadow Palette	FUSION FLY	₹562	Edit Delete
	Apple iPhone 17 Pro (Cosmic Orange, 512 GB)	Apple	₹144900	Edit Delete

12.Known Issues

The ShopEZ application is fully functional for its current scope; however, the following limitations have been identified during development:

- Online payment integration has not been implemented. The application currently supports order placement without processing real-time payments.
- Product search functionality supports basic keyword searching and can be enhanced with advanced filtering and sorting options.
- Product images are stored locally instead of using cloud storage, which may limit scalability for large deployments.
- Email notifications and SMS alerts for order confirmation are not available in the current version

13.Future Enhancements

The ShopEZ application can be further enhanced by implementing the following features:

- Integrate secure online payment gateways such as Razorpay or Stripe for real-time payment processing.

- Implement advanced product search, filtering, and sorting to improve the shopping experience.
- Add customer reviews and ratings for products.
- Introduce a wishlist feature that allows users to save products for future purchases.
- Send email and SMS notifications for order confirmation and delivery updates.
- Store product images using cloud storage services such as Cloudinary instead of local storage.
- Implement inventory management with automatic stock updates and low-stock alerts.